CS 437: Internet of Things
Lab 4: Cloud Infrastructure
Self-Driving Car - Cloud

Xabier Albizu (xabiera2) Ulises Díez (ulisesd2)
Ignacio Felices (jif4) Luis García (luis11)

April 14, 2026

Deliverables and Links

- **Team Members:** Xabier Albizu, Ulises Díez, Ignacio Felices, Luis García
- **Demo Video:** Link to Video
- **Github Repository:** Link to Repository

1 Introduction and Design Overview

In this laboratory, we developed a cloud-based infrastructure to support IoT operations for vehicle emissions monitoring. We leveraged Amazon Web Services (AWS) IoT platform to build our infrastructure, specifically using AWS IoT Core, Lambda, S3, Glue, and Athena.

Our system emulates 5 vehicles (out of a possible 12 configured IoT Things) that send real-time emission data to the cloud. A Lambda function processes this data, calculates the maximum CO2 emission for each vehicle, stores the data in S3, and sends results back to the devices. We then use AWS Glue to catalog the data and create visualizations to analyze emission patterns.

1.1 Architecture

The data flow follows this pattern:

```
Vehicle Emulator -> AWS IoT Core -> Lambda Function -> S3 Storage
                          |                                |
                    MQTT Response                     Glue Crawler
                          |                                |
                   Vehicle Display                    Athena Query
                                                           |
                                                     Visualizations
```

We chose this architecture because:

- **AWS IoT Core** provides a managed MQTT broker for device communication
- **Lambda** offers serverless processing without managing servers
- **S3** gives durable, scalable storage for all emission records
- **Glue** automatically catalogs our data for SQL queries
- **Athena** allows us to query data using standard SQL

2 Implementation

2.1 Building the Cloud Infrastructure

We created 12 AWS IoT Things (Car_0000 through Car_0009, plus two auxiliary devices) and organized them into a Thing Group called "Lab4Cars". Each Thing has:

- X.509 certificates for secure authentication
- A policy allowing MQTT connect, publish, subscribe, and receive operations
- Individual certificate files stored in the `certs/` directory

Our IoT endpoint is: `a1beiz43zcgzks-ats.iot.us-east-2.amazonaws.com`

We configured the emulator to use 5 vehicles simultaneously, each with its own MQTT client and CSV data file containing emission readings.

2.2 Device Emulator

The emulator (`lab4_emulator_client.py`) connects 5 vehicles to AWS IoT Core using the MQTT protocol. Each vehicle:

- Publishes emission data to the topic: `cars/emission`
- Subscribes to results on: `cars/result/Car_XXXX`
- Loads data from CSV files (`vehicle_data/vehicle0.csv` through `vehicle4.csv`)
- Sends readings when the user presses 's'

Each message contains:

- Device ID (e.g., Car_0000)
- Timestamp
- CO2 reading (the main metric we're monitoring)
- Additional data: speed, RPM, throttle position, engine temperature, and other emissions (CO, HC, O2, NOx)

2.3 Data Processing with Lambda

We created a Lambda function called `ProcessCarEmissions` that:

1. Receives the emission message from IoT Core (triggered by an IoT Rule)
2. Extracts the device ID and CO2 reading
3. Calculates and tracks the maximum CO2 for each vehicle (stored in memory)
4. Saves the complete record to S3 in JSON format
5. Publishes the result back to the device's specific topic

The Lambda function uses Python 3.12 and has the following IAM permissions:

- `AWSIoTDataAccess` - to publish results back to devices
- `AmazonS3FullAccess` - to write data to S3
- `AWSLambdaBasicExecutionRole` - for CloudWatch logging

Why we designed it this way: Initially, we tried using AWS Kinesis Firehose for streaming data to S3, but our account required subscription verification. We pivoted to having Lambda write directly to S3, which actually simplified our architecture and reduced latency.

2.4 Data Storage in S3

All emission data is stored in the S3 bucket `lab4-components-luisgarcia` with the following organization:

```
emissions-data/
  2026/
    04/
      14/
        21/
          Car_0000_1776200640687.json
          Car_0001_1776200640724.json
          ...
```

This hierarchical structure (year/month/day/hour) makes it easy for AWS Glue to partition the data and speeds up queries in Athena.

2.5 Data Cataloging with AWS Glue

We set up AWS Glue to automatically catalog our S3 data:

1. Created a database: `car_emissions_db`
2. Created a crawler: `car-emissions-crawler`
3. The crawler scanned our S3 bucket and automatically detected:

- 15 columns (device_id, timestamp, vehicle_co2, max_co2, etc.)
- Correct data types (doubles for numeric fields, strings for text)
- Partitioning by date/time hierarchy

4. This created a table: `emissions_data`

Now we can query the data using SQL in Amazon Athena, like:

```
SELECT device_id, AVG(vehicle_co2)
FROM emissions_data
GROUP BY device_id
```

3 Data Visualization

We downloaded the emission data from S3 and created visualizations using Python (Pandas, Matplotlib, Seaborn). The script `lab4_visualization.py` generates four charts:

3.1 Visualization 1: Average CO2 by Vehicle

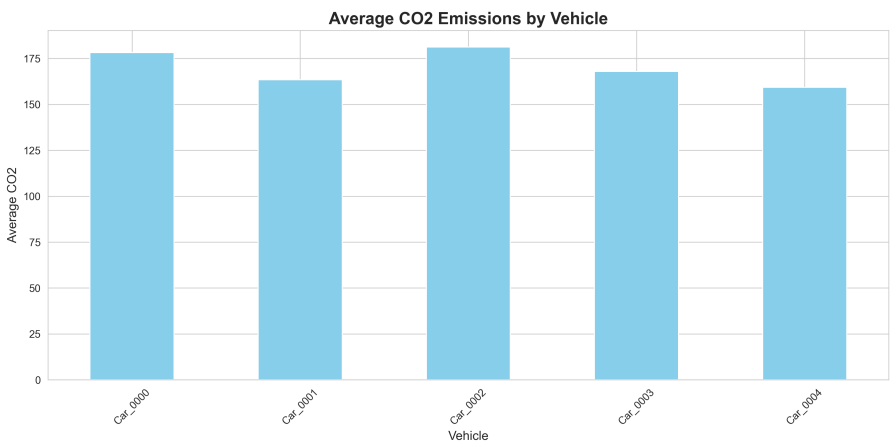

Figure 1: Bar chart showing the average CO2 emissions for each vehicle. Car_0002 has the highest average emissions.

This chart helps identify which vehicles are producing more emissions on average. In a real fleet, this could indicate vehicles needing maintenance.

3.2 Visualization 2: Current vs Maximum CO2

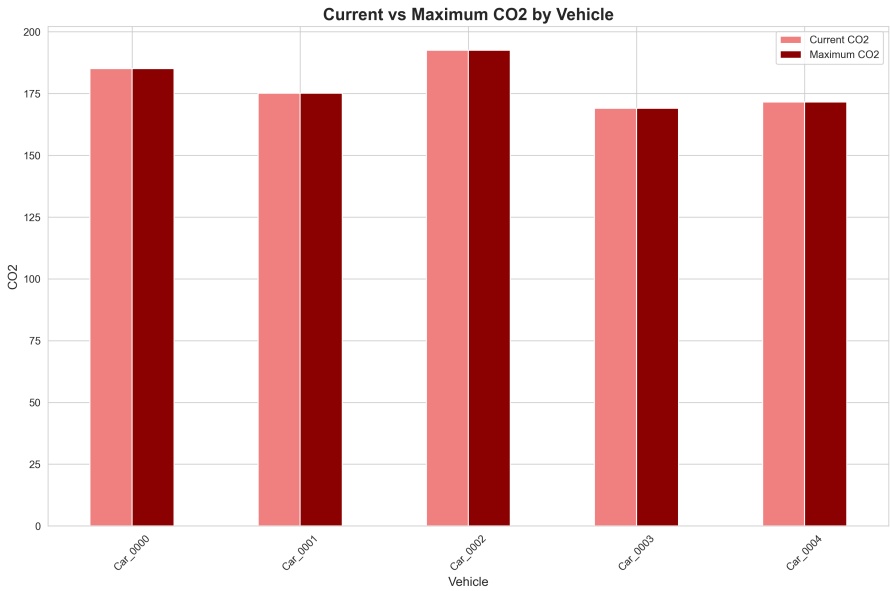

Figure 2: Comparison of current CO2 reading versus the maximum ever recorded for each vehicle.

This demonstrates that our Lambda function is correctly tracking and reporting the maximum CO2 for each vehicle.

3.3 Visualization 3: CO2 Distribution

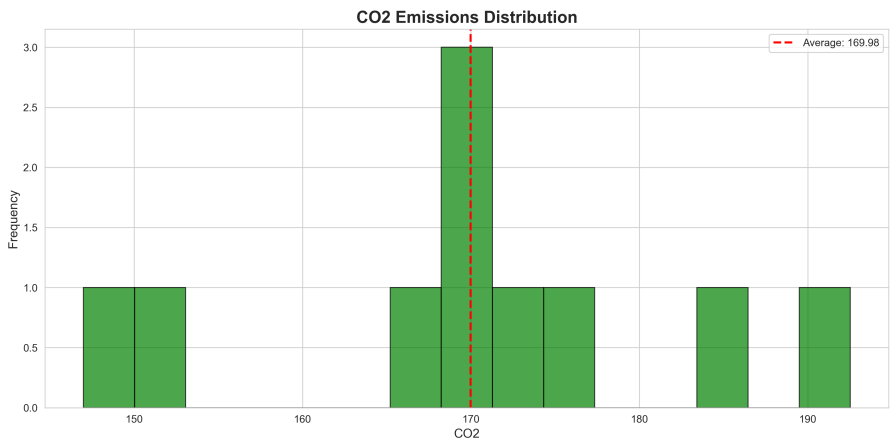

Figure 3: Histogram showing the distribution of all CO2 readings with average line at 169.98 ppm.

The distribution shows that most readings cluster around 170 ppm, with a range from about 147 to 193 ppm.

3.4 Visualization 4: CO2 Over Time

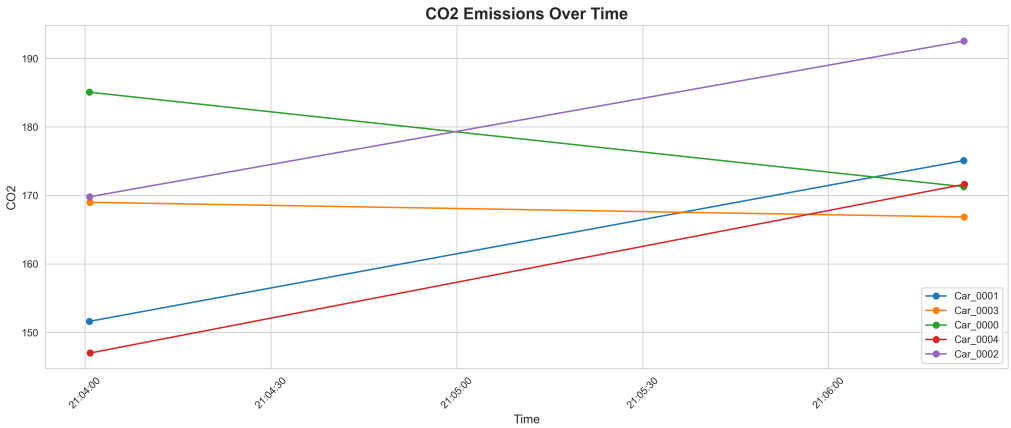

Figure 4: Timeline showing how CO2 emissions change over time for each vehicle. Each color represents a different car.

This temporal view shows trends in emissions. Some vehicles (like the green line) show decreasing emissions, while others (like the purple line) show increasing emissions over the observation period.

4 Results

Our system successfully:

- Connected 5 vehicles to AWS IoT Core using MQTT
- Processed 10+ emission messages in real-time
- Calculated maximum CO2 for each vehicle
- Stored all data persistently in S3
- Cataloged data automatically with AWS Glue
- Generated analytical visualizations

Performance metrics:

- Lambda execution time: 350ms per message
- Average CO2: 169.98 ppm
- CO2 range: 146.99 - 192.53 ppm
- Total data stored: 10 JSON records

5 Challenges and Solutions

Challenge 1: AWS Firehose Subscription

We initially planned to use AWS Kinesis Data Firehose to stream data to S3, but our account required subscription verification that was still pending. We solved this by having the Lambda function write directly to S3, which actually simplified our architecture.

Challenge 2: Column Name Case Sensitivity

The CSV data had mixed-case column names (like `vehicle_CO2`), but Pandas wasn't matching them correctly. We fixed this by normalizing all column names to lowercase in our visualization script.

Challenge 3: Lambda Timeout

The default 3-second timeout was too short for S3 writes. We increased it to 10 seconds, which gave plenty of time for the Lambda to complete all operations.

6 What We Learned

This lab taught us how to build a complete cloud IoT infrastructure. The most important concepts were:

- **MQTT Pub/Sub Model:** Understanding how devices publish data to topics and subscribe to receive results
- **Serverless Processing:** Using Lambda to process data without managing servers
- **Data Cataloging:** How AWS Glue automatically discovers schema and partitions
- **IAM Permissions:** Properly configuring roles so services can communicate
- **Data Pipeline Design:** Building a complete flow from device to visualization

7 Team Contributions

Note: The team collaborated equally on the design and troubleshooting. Specific areas of focus:

- **Xabier Albizu:** Set up AWS IoT Things and certificates. Configured the device emulator with MQTT clients.
- **Ulises Díez:** Developed the Lambda function for processing emissions. Created IoT Rules for triggering Lambda.
- **Ignacio Felices:** Configured AWS Glue database and crawler. Set up Athena for SQL queries.
- **Luis García:** Created the visualization pipeline. Generated all charts. Recorded the demo video and wrote this report.